

Practical No.8

Aim

To implement and demonstrate the RSA algorithm for encryption and decryption using modular arithmetic.

Theory

RSA Algorithm is a public-key cryptosystem that uses modular arithmetic for secure data transmission. The algorithm involves:

- **Key Generation:** Select two large prime numbers p and q , compute $n = p \times q$ and $(n) = (p-1) \times (q-1)$. Choose public key e such that $1 < e < (n)$ and $\gcd(e, (n)) = 1$. Calculate private key d such that $d \times e \equiv 1 \pmod{(n)}$.
- **Encryption:** Ciphertext $C = M^e \pmod{n}$, where M is the plaintext.
- **Decryption:** Plaintext $M = C^d \pmod{n}$, where C is the ciphertext.

Code

```
/**
 * RSA Algorithm Implementation - Practical 8
 * Test case: p=17, q=11, e=7, M=88, Expected: d=23, C=11
 */

// ===== MODULAR ARITHMETIC FUNCTIONS =====

function modExp(base: bigint, exp: bigint, mod: bigint): bigint {
  let result = 1n
  base = base % mod
  while (exp > 0n) {
    if (exp % 2n === 1n) result = (result * base) % mod
    exp = exp / 2n
    base = (base * base) % mod
  }
  return result
}
```

```
function gcd(a: bigint, b: bigint): bigint {
  while (b !== 0n) {
    const temp = b
    b = a % b
    a = temp
  }
  return a
}

function extendedGcd(
  a: bigint,
  b: bigint
): [bigint, bigint, bigint] {
  if (a === 0n) return [b, 0n, 1n]
  const [g, x1, y1] = extendedGcd(b % a, a)
  const x = y1 - (b / a) * x1
  const y = x1
  return [g, x, y]
}

function modInverse(a: bigint, m: bigint): bigint | null {
  const [g, x] = extendedGcd(a, m)
  if (g !== 1n) return null
  return ((x % m) + m) % m
}

// ===== KEY GENERATION =====

function generateRSAKeyPair(p: bigint, q: bigint, e: bigint) {
  const n = p * q
  const phi = (p - 1n) * (q - 1n)
  const d = modInverse(e, phi)

  return {
    publicKey: { n, e },
    privateKey: { n, d },
    phi
  }
}
```

```
// ===== ENCRYPTION & DECRYPTION =====
```

```
function rsaEncrypt(M: bigint, publicKey: { n: bigint; e: bigint }): bigint {  
    return modExp(M, publicKey.e, publicKey.n)  
}
```

```
function rsaDecrypt(C: bigint, privateKey: { n: bigint; d: bigint }): bigint {  
    return modExp(C, privateKey.d, privateKey.n)  
}
```

```
// ===== TEST CASE =====
```

```
console.log('=' .repeat(70))  
console.log('RSA ALGORITHM - PRACTICAL 8')  
console.log('=' .repeat(70))
```

```
// Test case parameters
```

```
const p = 17n  
const q = 11n  
const e = 7n  
const M = 88n  
const expectedD = 23n  
const expectedC = 11n
```

```
console.log('\n--- Test Case Parameters ---')  
console.log('Prime p = ' + p)  
console.log('Prime q = ' + q)  
console.log('Encryption key e = ' + e)  
console.log('Plaintext M = ' + M)
```

```
// Key generation
```

```
console.log('\n--- Key Generation ---')  
const keyPair = generateRSAKeyPair(p, q, e)  
console.log('n = p × q = ' + keyPair.publicKey.n)  
console.log('(n) = (p-1) × (q-1) = ' + keyPair.phi)  
console.log('e = ' + keyPair.publicKey.e)  
console.log('d = ' + keyPair.privateKey.d)  
console.log('Verification: d × e mod (n) = ' + ((keyPair.privateKey.d * keyPair.publi
```

```
// Verify decryption key
console.log('\n--- Verification ---')
if (keyPair.privateKey.d === expectedD) {
  console.log(' Decryption key verified: d = ' + keyPair.privateKey.d)
} else {
  console.log(
    ' Decryption key mismatch: expected ' +
    expectedD +
    ', got ' +
    keyPair.privateKey.d
  )
}

// Encryption
console.log('\n--- Encryption ---')
const C = rsaEncrypt(M, keyPair.publicKey)
console.log('C =  $M^e \bmod n$ ')
console.log('C = ' + M + '^' + keyPair.publicKey.e + ' mod ' + keyPair.publicKey.n)
console.log('C = ' + C)

// Verify ciphertext
if (C === expectedC) {
  console.log(' Ciphertext verified: C = ' + C)
} else {
  console.log(' Ciphertext mismatch: expected ' + expectedC + ', got ' + C)
}

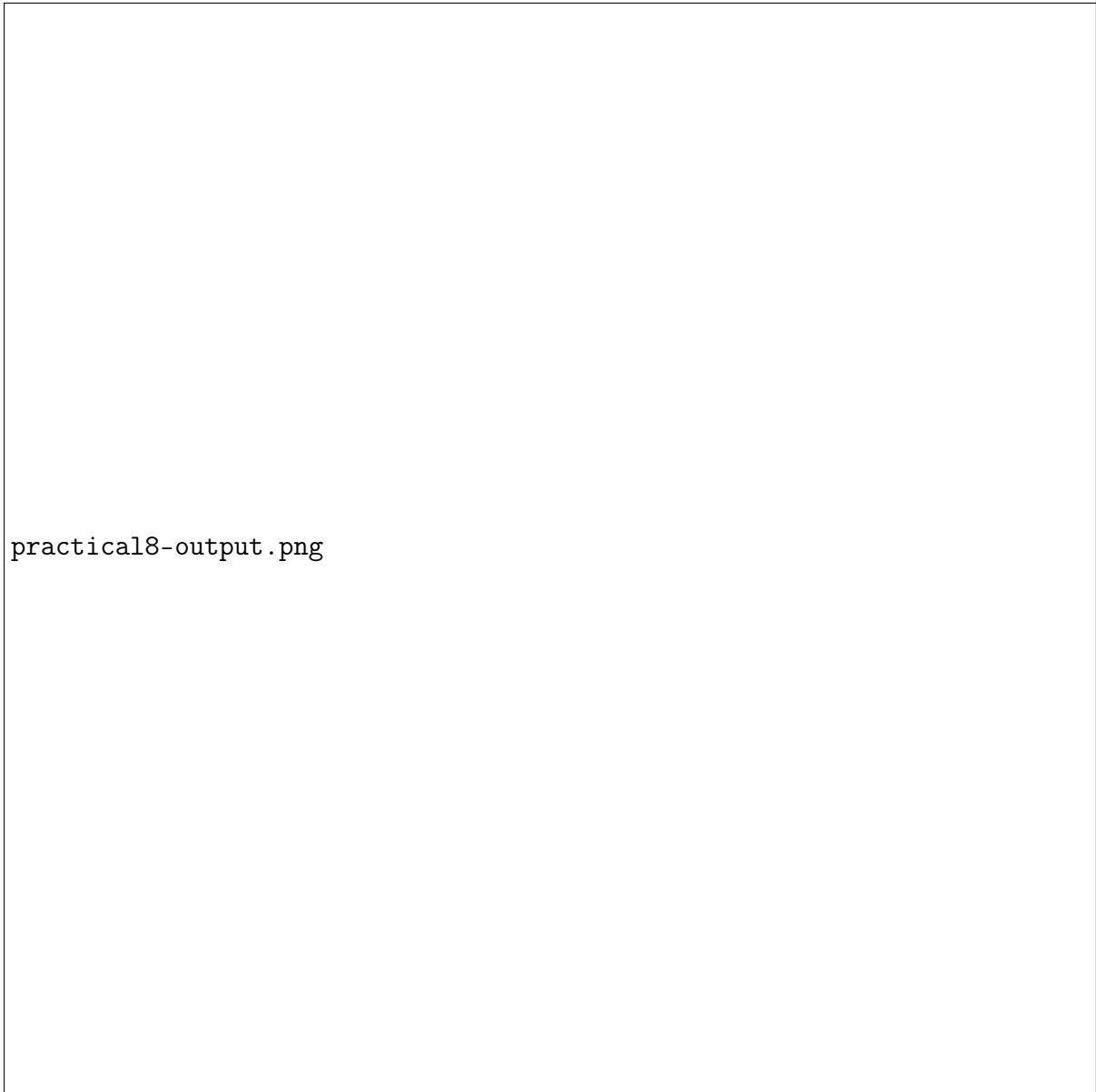
// Decryption
console.log('\n--- Decryption ---')
const decryptedM = rsaDecrypt(C, keyPair.privateKey)
console.log('M =  $C^d \bmod n$ ')
console.log('M = ' + C + '^' + keyPair.privateKey.d + ' mod ' + keyPair.privateKey.n)
console.log('M = ' + decryptedM)

// Verify decryption
if (decryptedM === M) {
  console.log(' Decryption verified: M = ' + decryptedM)
} else {
```

```
    console.log(' Decryption failed: expected ' + M + ', got ' + decryptedM)
  }

// Summary
console.log('\n' + '='.repeat(70))
console.log('SUMMARY')
console.log('=' .repeat(70))
console.log('Test case: ' + (C === expectedC && decryptedM === M ? 'PASSED ' : 'FAILED'))
console.log('All verifications: ' + (keyPair.privateKey.d === expectedD && C === expectedC ? 'PASSED ' : 'FAILED'))
console.log('=' .repeat(70))
```

Output



practical8-output.png

Figure 1: RSA Algorithm Implementation Output